

LLM7(GPT)

def. inspeção - rede (res. nominal, res. medida):
 Tolerância - obs = res. nominal $\times 0.9$
 Devia = (res. medida - res. nominal)
 Devia PCT = (Devia / res. nominal) $\times 100$

IF - Tolerância - obs $\leq$ devia $\leq$ Tolerância - obs:
 return "Sólo aprovado"

else:
 return (F"Seu valor: {MPa}"
 F"Devia: {Devia} MPa" Vn"
 F"Devia (PCT): {Devia PCT}%" F19%)")

def. controle velocidade (V atual, V alvo):
 IF V atual $\leq$ V alvo $\times 1.10$:
 return "Velocidade alta - risco de periclitamento - ignorar"

elif V atual $\leq$ V alvo $\times 0.90$:
 return "Velocidade baixa - perigo de redução"

else:
 return "Velocidade ok"

Testes
 Print("==== Teste Parte A ====")
 Print(f"inspeção - rede (120, 990)")
 Print(f"N==== Teste Parte B ====")
 Print(f"controle de velocidade (125, 120)")

1. Algoritmo conversão telemetria

// declaração de variáveis
 Indica t, dias, horas, minutos, segundos, resto1, resto2

// Início
 Estima "Digite o tempo total da medição em segundos:"

dia(t)
 // Processamento
 IF 3600 segundos: 24 horas *

3600 segundos
 dias $\leftarrow t / 86400$ // divisão
 valor para obter os dias
 resto1 $\leftarrow t - 86400$ // Resto que sobra dos dias (em segundos)

horas $\leftarrow resto1 / 3600$ // divisão inteira para obter os horas (em segundos)

resto2 $\leftarrow resto1 - 3600$ // Resto que sobra os horas (em segundos)

minutos $\leftarrow resto2 / 60$
 segundos $\leftarrow resto2 - 60$

Estima (Dias, "hor", "horas", ":", "minutos", ":", "segundos")

Fim

5. Pontuação

Parcial
 Encerra "Informe a resistência nominal (MPa):"
 Leia Resistencia - Nominal

Encerra "Informe a medida (MPa):"
 Leia Resistencia - Medida

Tolerancia = PCT $\times 8$
 Tolerancia - obs $\leq$ Resistencia nominal $\times 0.9$

Devia $\leftarrow$ Resistencia medida - Resistencia nominal
 Devia PCT $\leftarrow$ (Devia / Resistencia nominal) $\times 100$

Se Devia = - Tolerancia - obs $\leq$ Devia $\leq$ Tolerancia
 Encerra "Sólo aprovado"

Encerra "Devia, Devia, "MPa"
 Encerra "Devia (PCT), Devia PCT, "PCT"

Fim Se

// Parte B
 Encerra "Informe a velocidade dia (M/min):"
 Leia Velocidade - atual

Encerra "Informe a velocidade dia (M/min):"
 Leia Velocidade - atual

Se Velocidade - atual $>$ Velocidade dia $\times 1.10$ então
 Encerra "Velocidade alta - risco de periclitamento - ignorar"

Senão Se Velocidade - atual $\leq$ Velocidade dia $\times 0.90$ então
 Encerra "Velocidade baixa - perigo de redução"

Senão Encerra "Velocidade ok"

Fim

-- Parte A: Inspeção de Rede --
 res nominal = float(input("resistência nominal (MPa):"))
 res medida = float(input("resistência medida (MPa):"))

Tol - obs = res - nominal $\times 0.9$
 Devia = res medida - res nominal
 Devia PCT = (Devia / res nominal) $\times 100$

Print(f"N====> Resultados de Sólido")

IF - Tol - obs $\leq$ devia $\leq$ Tol - obs:
 Print(f"Sólo Sólido aprovado")

else:
 Print(f"Seu valor: {Devia} MPa"
 f"Devia PCT: {Devia PCT}%"

Parte B
 V - atual = float(input("V Velocidade atual (M/min):"))
 V - alvo = float(input("V Velocidade alvo (M/min):"))

Print(f"N====> Resultados de velocidade")

IF V atual $>$ V alvo $\times 1.10$:
 Print("Seu Velocidade alta - risco de periclitamento - ignorar")

elif V atual $\leq$ V alvo $\times 0.90$:
 Print("Seu Velocidade baixa - perigo de redução")

else:
 Print("Seu Velocidade ok")

LLM1 (CPT)

def chamfcar-Solo (SPT, esp, erg):

if SPT < 5 or esp < 50:

return "Inválida: Solo muito pouco - refusa mensagem"

else if SPT < 10 or esp < 50:

msg = "Refusa: fundação superficial não adequada"

if SPT < 10:

msg += "Menos: SPT abaixo do limiar 10"

if esp < 50 < 750:

msg += "Pouco: espessura de cimento inferior de 750mm"

else if SPT > 750 onde esp < 750 = 300

return "Euleria: Solo de alta capacidade"

else:

return "adequado: fundação adequada para"

Testes: [(3, 200), (25, 120), (35, 320)]

for SPT, esp in testes:

print(f"Para SPT={SPT}, Cap. Calc={cap}

RPa")

Print(chamfcar-Solo (SPT, cap))

LLM2 (Shoran)

R1 = float(input("R1 (Q): "))

R2 = float(input("R2 (Q): "))

R3 = float(input("R3 (Q): "))

VF = float(input("VF (V): "))

Rx = (R2 * R3) / R1

equilíbrio = (R1 + Rx) * (R2 + R3) / (R1 + Rx + R2 + R3)

T = total = VF + R * total

P = total = VF * 2 = total

equilíbrio = (R1 * Rx) = (R2 * R3)

Print(f"R1={R1}, R2={R2}, R3={R3}")

Print(f"Rx={Rx}, R1={R1}, R2={R2}, R3={R3}")

Print(f"R Total={R Total}, VF={VF}, A={A}")

Print(f"R Total={R Total}, VF={VF}, A={A}")

Print(f"R Total={R Total}, VF={VF}, A={A}")

Print(f"Equilíbrio={Equilíbrio}, Soma de equilíbrios={Soma}")

Exercício em Python:

LLM1 (Shoran):

t = int(input("Digite o tempo total de corrida em segundos"))

((t // 60) % 60) = minutos

minutos = minutos // 60

segundos = segundos % 60

Print(f"({minutos} dias, {horas} h, {minutos} min, {segundos} s)")

LLM2 (CPT):

t = int(input("Digite o tempo em segundos (>=0):"))

if t < 0:

return "Inválida: tempo negativo"

else:

Print(f"({minutos} dias, {horas} h, {minutos} min, {segundos} s)")